

Founding Engineer, Data Products — Take-Home + Live Defense

Time: aim for **3–4 hours**. Do not gold-plate. We care about your decisions and reasoning, not production polish.

Use any AI tool you like — we are AI-first and expect it. A model alone will often produce a confident, plausible, wrong answer. The judgment is yours. Honest reasoning and a flagged uncertainty beat a polished doc that guessed.

Context

At LH2 we turn raw enterprise data — Slack, Jira, GitHub, email, docs — into structured, privacy-safe, machine-trainable products we sell to AI labs. The core loop is:

Ingest → Normalize → Resolve identities → De-identify consistently → Assemble a sellable “decision” unit.

This task is a small slice of that job. The hard parts are: resolving who’s who across messy systems, de-identifying without breaking the data, and deciding what’s safe to sell.

The data

This folder is raw fragments from a fictional company, Acme. Real enterprise data is worse.

- `hr_directory.csv` — roster with a `team` column. Some rows are incomplete (missing `emp_id`, email, or handles).
- `slack.json` — messages from a few channels (visibility varies)
- `jira.json` — issues with assignees and comments
- `github.json` — PRs and commits with author metadata

What to build

1. Entity resolution (the core)

Map every real-world person to a single stable id, resolved across all sources. For each person, output:

- the surrogate id you assign (e.g. `PERSON_0001`)
- every raw identifier that maps to them (emails, names, handles, `emp_id`, github logins)
- how you matched each (authoritative? deterministic cross-system? fuzzy?) and your

confidence

If a mention is too weak to resolve, keep it unresolved and say why. Do not force a pick.

Short design note (max 1 page): your resolution approach and where it could be wrong.

2. Consistent de-identification

Produce a de-identified version of the data where every reference to a person — Slack body, Jira assignee, commit author line, free-text comment — is replaced by that person's surrogate, **consistently everywhere**. The same person must get the same surrogate across all systems.

Unresolved mentions should not be silently assigned to a guessed person.

3. Assemble one “decision unit”

Pick one real decision that plays out across the systems (e.g. a bug reported in Slack → tracked in Jira → fixed in a merged PR) and assemble it into a single structured object: sequence of events, people involved (as surrogates), citations back to source artifacts. Include only what is safe to sell.

4. Written note (max 1 page)

- The 2–3 hardest judgment calls you made and why
- Anything you decided was **not** safe to include in what we'd sell, and why
- What would break if this ran at 1000x scale

Deliverable

- ER mapping (JSON or a table)
- De-identified data (or transform code + a sample of output)
- The one assembled decision unit
- ≤2 pages of notes total (design + judgment calls)
- Working code where it makes sense — one hard step done for real beats everything stubbed

Live session (if we move forward)

45 minutes: you walk through the work, then we change the problem on you live — a new data quirk or constraint — and reason through it together. Come ready to defend choices and rethink them.